

# Manual técnico

---

Como a aplicação está feita, as decisões que não são óbvias a ler o código, e as armadilhas que já custaram bugs. Para a lista de endpoints, ver [API.md](#). Para instalar e correr, ver o [README](#). Para o manual de utilização, ver [MANUAL.md](#).

---

## 1. ARQUITECTURA

---

|                        |                                                            |
|------------------------|------------------------------------------------------------|
| web/                   | controllers + DTOs. Traduz HTTP para domínio e vice-versa. |
| service/               | regras de negócio. É aqui que vive a lógica.               |
| domain/                | entidades JPA e os cálculos que lhes pertencem.            |
| repository/            | interfaces do domínio + implementações em memória (testes) |
| repository/jpa         | adaptadores Spring Data (produção)                         |
| security/              | autenticação, autorização, filtro de conta activa          |
| resources/db/migration | Flyway. É o dono do esquema.                               |
| resources/static       | frontend, sem passo de build                               |

Stack: Java 21, Spring Boot 3.5, PostgreSQL, Flyway, HTML e JavaScript simples.

### O padrão dos repositórios

Os serviços dependem de **interfaces no pacote `repository`**, nunca de Spring Data. Há duas implementações de cada uma:

- `*RepositoryImpl` em memória, usada pelos testes, todas assentes em `RepositorioEmMemoria<T>`.
- Um adaptador em `RepositoriosJpa`, que delega num `*JpaRepository`.

Isto é o que permite os testes de serviço correrem sem base de dados nenhuma.

**Mantém as duas implementações com o mesmo comportamento**, incluindo a ordenação. Já houve fakes a devolver por ordem de inserção onde a consulta JPA real tinha `ORDER BY`, o que deixa um teste passar numa ordem que a produção não dá.

#### EntidadeBase

Os ids são atribuídos pela aplicação (`UUID.randomUUID()`), não pela base de dados. Sem `EntidadeBase` a implementar `Persistable`, o `save()` do Spring Data via um id preenchido, concluía que a entidade não era nova e fazia `merge` em vez de `persist`. As consequências eram silenciosas: relações em cascata não gravadas e a mesma entidade duas vezes na sessão.

`LigaLogo` é a excepção e não a estende, de propósito: é gravada por um `insert ... on conflict` nativo que nunca passa pelo `save()`.

---

## 2. MODELO DE DOMÍNIO

### Contas

`Gestor` é a única entidade com credenciais. `Treinador` não tem password: é alguém que treina uma equipa, criado pelo gestor.

`Treinador.conta` é um `@ManyToOne` opcional e **não único** para `Gestor`. Daí sai o que interessa:

- um treinador pode não ter conta nenhuma, e está tudo bem;
- a mesma conta pode treinar várias equipas, em ligas diferentes;
- quem gere uma liga e treina uma equipa usa um só login.

Há dois tipos de convite, com entidades separadas de propósito:

|         | CONVITE                                 | CONVITETREINADOR                        |
|---------|-----------------------------------------|-----------------------------------------|
| Cria    | administrador                           | gestor dono da liga                     |
| Para    | conta de gestor                         | ligar um treinador concreto a uma conta |
| Aberto? | sim, quem tiver o código escolhe quem é | não, já nasce apontado a um treinador   |

### `podeCriarLigas`

Só uma conta com esta flag cria ligas próprias. Nasce a `true` no caminho normal de convite de gestor, e a `false` em `TreinadorContaService.registar`. Está imposto no `SecurityConfig`:

```
.requestMatchers(HttpMethod.POST, "/api/ligas").hasAuthority("PODE_CRIAR_LIGAS")
```

`ligar()` (juntar um convite de treinador a uma conta que já existe) **não** mexe na permissão: quem já era gestor continua a ser.

A migração V6 criou a coluna com `default true`, por isso todas as contas anteriores ficaram com a permissão, incluindo contas de treinador. Não há código que corrija isso; é uma limpeza manual na administração.

### Jornadas

`EstadoJornada`: `ABERTA` → (`DESEMPATE`) → `FECHADA`. Só pode existir uma jornada não fechada por liga.

As primeiras cinco são `TREINO` e as seguintes `OFICIAL`, e **cada tipo numera a partir de 1**. É daí que vem a armadilha da secção 4.1.

### Dívidas

`Divida` tem uma equipa (um para um) e uma lista de `BlocoDivida`. Cada bloco é `INSCRICAO` (uma vez) ou `PERIODO` (recorrente), com um valor e um estado.

`RegraDivida` é opcional, uma por liga. Sem ela não há cobrança automática nenhuma. Com ela, o valor por posição é:

```
escalao = (posicao - 1) / equipasPorEscalao // divisão inteira
valor = min(valorInicial + incremento * escalao, valorMaximo)
```

O mesmo cálculo está espelhado em `calcularValorEscalao` no `app.js`, mas só para a pré-visualização. **A cobrança a sério é sempre do servidor.**

`Divida.proximoNumeroBloco` é persistido e protegido por `@Version`, para duas transacções em simultâneo não gravarem dois blocos com o mesmo número. Na prática quem perde a corrida esbarra primeiro na restrição `bloco_divida_numero_unico`, porque o Hibernate grava as inserções antes das actualizações; ambos os casos dão 409 pelo `GlobalExceptionHandler`.

---

## 3. OS DOIS FLUXOS QUE MEXEM EM DINHEIRO

---

### 3.1 Fecho de bloco

Em `JornadaService.fecharJornada`, depois de a jornada fechar:

1. Se a liga não tiver `RegraDivida`, não acontece nada.
2. Recolhem-se as jornadas **fechadas e ainda não incluídas num bloco** (`Jornada.incluidaEmBloco`).
3. Se forem tantas ou mais do que `jornadasPorBloco`, fecha-se um bloco.
4. Cada equipa activa paga a **soma do seu valor em cada uma dessas jornadas**, pela posição que teve em cada uma.
5. Essas jornadas ficam marcadas, e nunca mais entram noutro bloco.

O passo 5 é o que torna isto seguro. Sem essa marca, mudar `jornadasPorBloco` a meio da época fazia o fecho seguinte olhar outra vez para jornadas já cobradas e cobrá-las a dobrar. E como a selecção é por marca e não por "as últimas N", não depende de ordenação nenhuma.

### 3.2 Desempate

Se, ao fechar, sobraem equipas com a mesma pontuação, a jornada fica em `DESEMPATE`, **sem cobrar nada**.

`resolverDesempate(jornada, ordem)` aplica a ordem dada, atribui posições sequenciais, marca os resultados afectados com `desempateManual` e só então deixa correr o fecho do bloco.

A ordenação primária continua a ser a pontuação; a ordem submetida é só critério secundário, por isso uma lista mal formada nunca troca equipas entre pontuações diferentes. A validação exige que a lista seja exactamente o conjunto empatado.

`inserirResultado` recusa enquanto a jornada está em `DESEMPATE`. Sem isso, mexer nas pontuações invalidava o empate detectado, em silêncio.

---

## 4. ARMADILHAS

---

Todas estas já produziram bugs reais. Valem uma leitura antes de mexer.

### 4.1 `liga.getJornadas()` não vem por ordem cronológica

A colecção é `@OrderBy("numJornada")`, e `numJornada` reinicia em 1 quando começam as oficiais. Assim que uma liga tem os dois tipos, essa ordem intercala-os.

Usa `Jornada.ORDEN_CRONOLOGICA` sempre que a ordem real interessar (tipo primeiro, número depois). Não ordenes por tipo no `@OrderBy`: como o enum é gravado como texto, "OFICIAL" viria antes de "TREINO".

### 4.2 `listarDividas(liga, estado)` não serve para somar dinheiro

Filtra pelo estado da **dívida da equipa**, que só diz se ainda sobra alguma coisa por pagar. Uma dívida pendente pode ter blocos já pagos lá dentro.

Para dinheiro, usa `listarPorLiga(liga)` e soma **bloco a bloco**, como faz `calcularPoteDaLiga`.

### 4.3 O estado da dívida é calculado, nunca assumido

`Divida.atualizarEstado()` recalcula a partir dos blocos, e é chamado sempre que algum muda. Não ponhas `setEstado` à mão. Um bloco de valor zero nasce já resolvido; foi por se assumir o estado que uma dívida cujo primeiro bloco era de 0,00 € ficava pendente a dever zero.

### 4.4 As permissões da sessão são relidas a cada pedido

`GestorAutenticado` é uma fotografia tirada no login e vive na sessão. O `ContaAtivaFilter` relê a linha do gestor em cada pedido e **reconstrói o principal**, senão retirar uma permissão não tinha efeito enquanto a pessoa não saísse e voltasse a entrar.

O filtro isenta apenas os caminhos que não pressupõem sessão válida (login, registo, logout). Não voltes a isentar o prefixo `/api/auth/` inteiro: `/estado`, `/password` e `/treinador/ligar` usam uma sessão aberta como qualquer rota protegida.

### 4.5 No frontend, tudo o que muda dados chama `recarregar()`

Não há estado reactivo. Enquanto cada handler decidia o que refrescar, faltava sempre um pedaço: houve um painel a ficar com dados velhos e um mostrador a não mexer depois de um pagamento.

Existe um `recarregar()` (ligas, detalhe, e as dívidas só se a tab estiver aberta) e todas as acções que mudam alguma coisa chamam-no. Não acrescentes refrescamentos avulso.

### 4.6 Nunca ponhas `-Djava.net.preferIPv6Addresses=true`

Já estive no `Dockerfile`, posto para a rede privada do Railway, que só existe em IPv6. Estava a mais e fazia mal.

A opção só decide a **ordem** quando um nome tem endereços das duas famílias. Um nome que só tem IPv6, como o `postgres.railway.internal`, é usado pela JVM sem opção nenhuma. Medido, com e sem:

| PREFERIPv6ADDRESSES | API.RESEND.COM (AS DUAS) | NOME SÓ COM IPV6 |
|---------------------|--------------------------|------------------|
| true                | IPv6                     | IPv6             |
| system              | IPv4                     | IPv6             |
| ausente             | IPv4                     | IPv6             |

O que ela fazia era mandar para IPv6 tudo o que tem as duas, incluindo a API do Resend. O Railway não tem saída IPv6 para a internet pública, por isso os emails de recuperação morriam em `Network is unreachable`.

O sintoma apareceu longe da causa: parecia problema do Resend, ou do DNS do domínio, e era uma opção da JVM posta meses antes por outra razão. Se um dia alguma chamada a um serviço externo falhar com `Network is unreachable` em produção, começa por aqui.

## 5. SEGURANÇA

- **Autenticação** por sessão, com cookie `HttpOnly` e `SameSite=Lax`. Passwords em BCrypt.
- **Autorização por dono, não por perfil.** Quase tudo é isolado pelas consultas: `buscarPorIdEGestor` faz a verificação acontecer dentro do SQL. As únicas excepções por papel são `/api/admin/**` (`ROLE_ADMIN`) e criar liga (`PODE_CRIAR_LIGAS`).
- **CSRF** com cookie `XSRF-TOKEN` legível por JavaScript, reenviado no cabeçalho `X-XSRF-TOKEN`. Obrigatório em tudo o que não seja `GET`.
- **404 em vez de 403** quando o recurso é de outra pessoa, para não confirmar que existe.
- **Contas desactivadas** são barradas no pedido seguinte, não no próximo login.
- **Sessões cortadas por mudança de password.** `Gestor.sesoesValidasDesde` é uma data de corte: o `ContaAtivaFilter` recusa qualquer sessão criada antes dela. Sem isto, mudar a password não expulsava ninguém, porque a sessão vive do lado do servidor e não sabe nada da password. Quem recupera a password costuma fazê-lo por desconfiar que alguém entrou; deixar a sessão dessa pessoa viva tornava a recuperação inútil. Também é accionada quando um administrador corrige o email de uma conta.
- **Recuperação de password** (`RecuperacaoService`): pedir responde sempre 204, exista a conta ou não, para o endereço não servir de lista de quem está registado. O código tem 24 bytes aleatórios, vale uma hora, serve uma vez, e é guardado em SHA-256 e não em claro. Máximo de três pedidos por conta por hora.
- **Logos** são validados pelos primeiros bytes (`ImagemSuportada`), não pelo `Content-Type` que o cliente declara. SVG fica de fora, para não servirmos scripts do nosso próprio domínio.

## 6. BASE DE DADOS

Flyway é o dono do esquema e o Hibernate está em `ddl-auto=validate`: se as entidades não corresponderem às tabelas, a aplicação não arranca.

| MIGRAÇÃO | O QUE FAZ                                                            |
|----------|----------------------------------------------------------------------|
| V1       | esquema inicial: gestor, treinador, liga, equipa, jornada, resultado |
| V2       | administração e convites                                             |
| V3       | logo por liga                                                        |
| V4       | contas de treinador                                                  |
| V5       | dívidas, blocos e regra                                              |
| V6       | permissão de criar ligas                                             |
| V7       | marca de jornada já incluída num bloco                               |

**Nunca edites uma migração já aplicada em produção.** O Flyway guarda o checksum e recusa arrancar se ele mudar. Acrescenta uma migração nova.

Antes de aplicar uma migração em produção, faz cópia e confirma que ela é legível:

```
railway connect Postgres --tunnel-only --port 15432
pg_dump "postgresql://..." -Fc -f backup.dump
pg_restore --list backup.dump | head
```

Nada apaga ligas nem equipas: quem sai fica marcado como desistente e o histórico mantém-se.

## 7. FRONTEND

Sem passo de build, sem framework, sem dependências externas. A Content-Security-Policy é `default-src 'self'` e até os tipos de letra são servidos pela aplicação, para não haver excepções.

**Cada página duplica os seus próprios utilitários** (`api()`, `mostrarAlerta()`, `tokenCsrft()`, `texto()`). É uma decisão, não um esquecimento: sem bundler, partilhar significaria mais um pedido e mais uma ordem de carregamento para acertar.

Escapa sempre o que vem do servidor com `texto()` antes de o meter em HTML.

Páginas: `index.html` (gestor), `minhas-ligas.html` e `minhas-dividas.html` (treinador), `admin.html`, `login.html`, `registo.html`, `registo-treinador.html`, `conta.html`.

### Folha de estilos

Tema escuro, com um sistema de tokens no topo do `styles.css` (cores, escala tipográfica, sete valores de espaçamento). Duas famílias, Archivo e Archivo Narrow.

Há **um só acento**, o laranja `--sinal`, e usa-se apenas onde carrega informação: o líder da classificação, o separador activo e o foco do teclado. Não o uses para mais nada, ou deixa de querer dizer alguma coisa. Para estados há cores próprias: `--ok` verde, `--espera` âmbar, `--fim` vermelho, `--frio` azul.

---

## 8. TESTES

```
mvn test
```

126 testes, todos ao nível do serviço ou do domínio, com os repositórios em memória. **Não há testes de controller**, por convenção: a lógica está nos serviços e é lá que é testada.

`ArranqueTest` levanta o contexto Spring com Testcontainers, por isso precisa de Docker. Para correr sem ele:

```
mvn test -Dtest='!ArranqueTest'
```

### Como testar em condições

A base de dados de desenvolvimento pode ter credenciais desalinhadas com o `.env`, porque a password só é aplicada quando o volume é criado. Para provar comportamento ponta a ponta, o mais rápido é levantar um ambiente descartável:

```
docker run -d --name qa-db -e POSTGRES_DB=qa -e POSTGRES_USER=qa \
  -e POSTGRES_PASSWORD=qa123 -p 55432:5432 postgres:18

DB_URL=jdbc:postgresql://localhost:55432/qa DB_USER=qa DB_PASSWORD=qa123 \
  ADMIN_EMAIL=qa@teste.local ADMIN_PASSWORD=qalocal12345 mvn spring-boot:run
```

Vale a pena: bugs de refrescamento da interface e de estado só aparecem assim.

---

## 9. DEPLOY

`master` faz deploy automático no Railway. O jar é único e serve também o frontend.

```
railway status          # estado do serviço
railway logs            # ver o arranque
```

Confirma sempre nos logs que as migrações validaram e que o arranque foi limpo. Confirma também que não aparece o aviso `Sem EMAIL_CHAVE ou EMAIL_REMETENTE`: se aparecer, os emails de recuperação vão para o log em vez de serem enviados, e ninguém consegue recuperar a password.

### Email

Três variáveis de ambiente:



| VARIÁVEL        | PARA QUE SERVE                              |
|-----------------|---------------------------------------------|
| EMAIL_CHAVE     | Chave de API do Resend                      |
| EMAIL_REMETENTE | Remetente, num domínio verificado no Resend |
| APP_URL         | Endereço público, usado no link do email    |

O `EMAIL_REMETENTE` tem mesmo de ser de um domínio verificado no Resend, com os registos de DNS configurados. Com o remetente de teste deles, só a caixa de correio do dono da conta recebe as mensagens, o que na prática deixa a recuperação sem funcionar para toda a gente menos uma pessoa.

Sem chave nenhuma a aplicação arranca à mesma e o `EnviadorParaLog` escreve as mensagens na consola, com o link e tudo. É assim que o fluxo se experimenta em desenvolvimento.

## O HTML das mensagens

Está todo no `ModeloDeEmail`, e parece antiquado de propósito. HTML de email não é HTML de página: tabelas para a disposição, estilos em linha, sem `<style>`, sem imagens e sem os tipos de letra da aplicação, porque um `@font-face` não carrega em cliente de email nenhum. O botão é uma célula de tabela com cor de fundo e não um `<a>` com padding, senão o Outlook, que desenha com o motor do Word, mostrava um link azul sublinhado onde devia estar um botão laranja.

Cada mensagem leva sempre as duas versões, texto e HTML. Quem recebe em texto simples tem de conseguir recuperar a password na mesma, e um email só com HTML é olhado de lado pelos filtros de spam.

O nome de quem recebe é escrito pela própria pessoa no registo e vai para dentro de HTML, por isso é escapado. Há um teste para isso.

O túnel para a base de dados ( `railway connect Postgres --tunnel-only` ) falha com alguma frequência e não há URL pública alternativa. Quando isso acontecer, reproduzir localmente costuma responder melhor à pergunta do que espreitar os dados de produção.

## 10. POR IMPLEMENTAR

- Verificação de email no registo. O convite já trava o registo aberto, que é o que a confirmação costuma proteger; o que fica por resolver é o email mal escrito, e para esse o remendo é o administrador corrigi-lo à mão.
- Limitação de tentativas de login.
- Desempate na classificação geral (hoje ordena por nome).
- Reabrir uma jornada fechada.